

Assignment 2: Coding Basics

Morgan Finch

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.  
seq5 <- seq(1, 55, 5) # from 1 to 55 by increments of 5  
seq5
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
#2.  
mean_seq5 <- mean(seq5) #mean is 26  
med_seq5 <- median(seq5) #median is 26
```

```
#3.  
greater <- mean_seq5 > med_seq5  
greater #False, mean is not greater than the median
```

```
## [1] FALSE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
vec_names <- c("Morgan", "Clare", "Sophi", "Andrea") #create vector for student names  
vec_names #return vec
```

```
## [1] "Morgan" "Clare" "Sophi" "Andrea"
```

```
class(vec_names) #class: character
```

```
## [1] "character"
```

```
vec_scores <- c(95, 92, 98, 77) #create vector for test scores  
vec_scores
```

```
## [1] 95 92 98 77
```

```
class(vec_scores) #class: numeric
```

```
## [1] "numeric"
```

```
vec_scholar <- c(FALSE, FALSE, TRUE, FALSE) #create vector for scholarship or not  
vec_scholar
```

```
## [1] FALSE FALSE TRUE FALSE
```

```
class(vec_scholar) #class: logical
```

```
## [1] "logical"
```

```
##create a dataframe  
dataframe1 <- data.frame(vec_names,vec_scores,vec_scholar)  
is.data.frame(dataframe1) #confirm dataframe
```

```
## [1] TRUE
```

```
#Set column names  
names(dataframe1) <- c("Student Names","Test Scores","Scholarship Status")
```

9. QUESTION: How is this data frame different from a matrix?

Answer: Data frames and matrices are both 2 dimensional structures. Data frames, however, are more general than a matrix and can have different element. A matrix has elements of the same type.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else
ifelse_1 <- function(x) {
  if(x > 50) { #if greater than 50
    x="Pass"   #print pass
  }
  else {
    x="Fail"   #else print fail
  }
}
less50 <- ifelse_1(20); less50 #20 is less than 50, Fail
```

```
## [1] "Fail"
```

```
greater50 <- ifelse_1(60); greater50 #60 is greater than 50; Pass
```

```
## [1] "Pass"
```

```
#11. Create a function using ifelse()
ifelse_2 <- function(x){
  ifelse(x>50, "Pass", "Fail")
}
value20 <- ifelse_2(20); value20 #fail <50
```

```
## [1] "Fail"
```

```
value75 <- ifelse_2(75); value75 #pass >50
```

```
## [1] "Pass"
```

```
#12a. Run the first function with the value 52.5
print(ifelse_1(52.5)) #Pass
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5  
print(iffelse_2((52.5))) #Pass
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores  
###value_1 <- iffelse_1(vec_scores);value #Error in if (x > 50) { : the condition has length > 1
```

```
#13b. Run the second function with the vector of test scores  
value_2 <- iffelse_2(vec_scores); value_2 #"Pass" "Pass" "Pass" "Pass"
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `iffelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: The `if...else` function returned the error `if (x > 50) { : the condition has length > 1`. The `if...else` function can only run for one element at a time. The `iffelse()` function was able to compute each component in the vector.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)